Kraft-McMillan Inequality (KMI)

Presentation by:
Joe Louis Paul I

Kraft-McMillan Inequality (1/2)

- If a prefix code C has been constructed for a discrete memoryless source (DMS) with source alphabet $S=\{s_0, s_1, \ldots\ldots\ldots, s_{K-1}\}$ and source statistics $P=\{p_0, p_1, \ldots\ldots\ldots, p_{K-1}\}$ and the codeword for symbol s_k has length l_k, for $k=0,1,\ldots\ldots\ldots,K-1$, then the codeword lengths of the code C always satisfy a certain inequality known as the ***Kraft-McMillan Inequality (KMI)***.

Kraft-McMillan Inequality (2/2)

- The KMI Theorem is shown by,

 # The three codebooks and their lengths

Let's list the codes (taken from the image) and compute lengths l_k:

C1

- S0 → `0` → $l_0 = 1$
- S1 → `01` → $l_1 = 2$
- S2 → `011` → $l_2 = 3$
- S3 → `0111` → $l_3 = 4$

C2

- S0 → `0` → $l_0 = 1$
- S1 → `10` → $l_1 = 2$
- S2 → `110` → $l_2 = 3$
- S3 → `111` → $l_3 = 3$

C3

- S0 → `0` → $l_0 = 1$
- S1 → `1` → $l_1 = 1$
- S2 → `00` → $l_2 = 2$
- S3 → `11` → $l_3 = 2$

Now compute $\sum 2^{-l_k}$ for each column — we'll brea ch step down.

$$\sum_{k=0}^{K-1} 2^{-l_k} \leq 1$$

Where, 2 is the radix (i.e. number of symbols) involved binary alphabet {0,1}.

Computing the Kraft sums (step-by-step)C1Compute 2−□2−l term by term:2−1=1/2=0.52−1=1/2=0.52−2=1/4=0.252−2=1/4=0.252−3=1/8=0.1252−3=1/8=0.1252−4=1/16=0.06252−4=1/16=0.0625Add them:0.5+0.25+0.125+0.0625=0.9375.0.5+0.25+0.125+0.0625=0.9375.So for C1 the sum is 0.9375 (≤1≤1).C2Term by term:2−1=0.52−1=0.52−2=0.252−2=0.252−3=0.1252−3=0.1252−3=0.1252−3=0.125 (since S2 and S3 both have length 3)Add:0.5+0.25+0.125+0.125=1.0.0.5+0.25+0.125+0.125=1.0.So C2 gives sum 1.0 (exact equality).C3Term by term:2−1=0.52−1=0.52−1=0.52−1=0.52−2=0.252−2=0.252−2=0.252−2=0.25Add:0.5+0.5+0.25+0.25=1.5.0.5+0.5+0.25+0.25=1.5.So C3 gives 1.5, which is > 1.

Example : KMI Theorem (1/2)

- Find out the $\sum_{k=0}^{K-1} 2^{-l_k}$ for C_1, C_2 and C_3.

Probabilities: $p_0 = 0.5,\ p_1 = 0.25,\ p_2 = 0.125,\ p_3 = 0.125.$

Entropy:

$$\begin{aligned} H &= -\sum_k p_k \log_2 p_k \\ &= -(0.5\log_2 0.5 + 0.25\log_2 0.25 + 0.125\log_2 0.125 + 0.125\log_2 0.125) \\ &= -(0.5\cdot(-1) + 0.25\cdot(-2) + 0.125\cdot(-3) + 0.125\cdot(-3)) \\ &= 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits.} \end{aligned}$$

So the theoretical lower bound on average length is **1.75 bits per symbol.**

Compute average length $L = \sum_k p_k l_k$ for each code:

- **C1:** $L = 0.5\cdot 1 + 0.25\cdot 2 + 0.125\cdot 3 + 0.125\cdot 4$
  $= 0.5 + 0.5 + 0.375 + 0.5 = 1.875$ bits per symbol.
  So C1: $L = 1.875$. Redundancy $L - H = 0.125$ bits.
- **C2:** $L = 0.5\cdot 1 + 0.25\cdot 2 + 0.125\cdot 3 + 0.125\cdot 3$
  $= 0.5 + 0.5 + 0.375 + 0.375 = 1.75$ bits per symbol.
  C2: $L = H$. This is optimal for this discrete distribution — because the probabilities are exact dyadic powers of 1/2 (0.5, 0.25, 0.125, 0.125), so you can reach the entropy with integer lengths.
- **C3:** $L = 0.5\cdot 1 + 0.25\cdot 1 + 0.125\cdot 2 + 0.125\cdot 2$
  $= 0.5 + 0.25 + 0.25 + 0.25 = 1.25$ bits per symbol.
  But wait — **1.25 < H**. That's impossible for a valid uniquely decodable code (you can't beat entropy).
  That paradox flags the earlier check: **C3 is not uniquely decodable**. The average-length calculation is meaningless for a code that allows ambiguous decodings.

So the only fully valid, instantaneous, and optimal code among these is **C2**.

Source Symbol	Probability	C_1	C_2	C_3
S0	0.5	0	0	0
S1	0.25	01	10	1
S2	0.125	011	110	00
S3	0.125	0111	111	11

A few concrete decoding examples

- **C2** (instantaneous): codebook `{0, 10, 110, 111}`.
  Given bitstream `1101110`, greedy decoding reads:
  - `110` → S2
  - remaining `111` → S3
  - remaining `0` → S0
  So decoded *symbols*: S2, S3, S0 — easy peasy.
- **C1** (not instantaneous but uniquely decodable): `{0, 01, 011, 0111}`. Decoding requires checking until you match one of the codewords. Example: bitstream `010111`:
  - read `0`: `0` is a codeword (S0), but also other longer codewords start with `0`. If you choose immediate greedy decode to S0, the remainder `10111` would not parse into valid codewords because none start with `1`. So the correct parse is to read until you hit the shortest *complete* match that makes the entire stream decodable. In practice a decoder can use buffering and a finite-state machine to ensure unique reconstruction.
- **C3**: ambiguous: the bitstring `00` could mean S2 (`00`) or S0+S0 (`0 0`). So you can't reliably recover the original symbol sequence.

Visual intuition (binary tree view)

- **Prefix codes** correspond to choosing leaves in a binary tree; no chosen leaf is an ancestor of another chosen leaf. Each leaf at depth ℓ consumes $2^{-\ell}$ of the "tree capacity." The KMI sum is the fraction of the tree used.
  - C2 with sum = 1 fills the available capacity exactly (a complete packing).
  - C1 with sum = 0.9375 fills most of the tree but uses some internal nodes as codewords (so it's not prefix-free); it still fits.
  - C3 with sum = 1.5 would require more than 100% of the tree — impossible — so you must have overlaps/prefixes that break unique decodability.

Quick checklist / takeaways

- If $\sum 2^{-l_k} > 1$: impossible to make a uniquely decodable binary code with those lengths (C3).
- If $\sum 2^{-l_k} \le 1$: a prefix code may exist with those lengths (C1 and C2 pass this test).
- Prefix-free implies instantaneous and is easier to decode. Unique decodability is weaker; a code can be uniquely decodable but not prefix-free (C1).
- Average length L must satisfy $L \ge H$ for any uniquely decodable code. If you find $L < H$, something's wrong — the code is not uniquely decodable (C3).
- Equality $L = H$ is possible for dyadic probabilities (powers of 1/2); that's exactly what C2 achieves here.

Interpretation: what these sums tell us about the codes

- **C2 (sum = 1.0):** This is a *perfect* prefix code. None of its codewords is a prefix of another (check: `0`, `10`, `110`, `111` —no one is a prefix of another), so it's **prefix-free (instantaneous)**. The Kraft sum being exactly 1 means the binary tree is exactly full for these leaf depths — no leftover capacity. Decoding can be done greedily, bit by bit, with no lookahead.
- **C1 (sum = 0.9375 ≤ 1):** The sum is ≤ 1, so a uniquely decodable code with those lengths could exist — and indeed the listed codewords are **uniquely decodable**, but **not prefix-free**: `0` is a prefix of `01`, `011`, `0111`. That means you can't always decode by the simplest greedy rule (stop immediately when you see a codeword) — you may need to read a little further to be sure. Still, McMillan's theorem allows such a code: the inequality holds, and no ambiguous concatenation exists for this particular set of codewords. The code is *not* instantaneous, but it is uniquely decodable.
- **C3 (sum = 1.5 > 1):** This violates Kraft–McMillan. That means **no uniquely decodable binary code** can have lengths $\{1, 1, 2, 2\}$. In practice, the listed codebook is indeed ambiguous: `00` could be parsed as symbol S2 (code `00`) *or* as S0 (`0`) followed by S0 (`0`) — two different parses of the same bitstring. So **C3 is invalid** as a uniquely decodable code.

Q: Why can C1 be uniquely decodable even though it's not prefix-free?
A: Prefix-free is a *sufficient* condition for unique decodability, but not *necessary*. C1 is carefully constructed so that no concatenation of codewords produces two different sequences of source symbols — even though some codewords are prefixes of longer codewords. That makes it uniquely decodable but not instantaneous.

Q: Is Kraft's inequality both necessary and sufficient?
A: For prefix codes: yes — Kraft's inequality is necessary and sufficient for the existence of a prefix code with those lengths. For uniquely decodable codes, Kraft–McMillan provides a necessary condition (the sum ≤ 1). So if you have lengths that satisfy Kraft, a prefix (instantaneous) code exists. If the lengths violate Kraft, no uniquely decodable code exists.

Q: How do I design an optimal code for a given distribution?
A: Huffman coding produces a prefix-free code that minimizes expected length L for a given discrete distribution. If the probabilities are exact negative powers of 2, Huffman yields $L = H$.

Conclusion

That slide packs a lot into a small picture: it's a compact demonstration of the **Kraft–McMillan inequality**, the difference between prefix-free and merely uniquely decodable codes, and the practical check you can do to see whether a proposed code is even possible. Bottom line:

- **C2** is the clean winner: prefix-free, instantaneous, Kraft sum = 1, average length = entropy (optimal).
- **C1** passes Kraft's test (0.9375 ≤ 1) and is uniquely decodable but **not** instantaneous — so it's decodable but you need a non-greedy decoder.
- **C3** violates Kraft (1.5 > 1) and is therefore **invalid** (ambiguous decoding example: `00`).

Example : KMI Theorem (2/2)

- Take C_1, $\sum_{k=0}^{K-1} 2^{-l_k} = 0.9375 < 1$
- Take C_2, $\sum_{k=0}^{K-1} 2^{-l_k} = 1$
- Take C_3, $\sum_{k=0}^{K-1} 2^{-l_k} = 1.5 > 1$
- The coding schemes C_1 and C_2 satisfy the Kraft-McMillan Inequality.
- *Though, C_1 satisfies KMI, but its not a preix code (But fully reversible).*

Important Note on KMI (1/2)

- The Kraft-McMillan Inequality <u>forces the condition on the length of the codewords of a coding scheme (code) rather than controls the code itself</u>.
- Or in other words,

  <u>*Kraft-McMillan Inequality does not make a code as a prefix code*</u>

Important Note on KMI (2/2)

- From the example,
  - Code III violates the Kraft-McMillan Inequality; it can not therefore be a prefix code.
  - Kraft-McMillan Inequality is satisfied by both codes I and II; but only code II is a prefix code.

Prefix Codes as Instantaneous Codes

- Prefix codes are distinguished from other uniquely decodable codes by the fact that the *end of a codeword is always recognizable*.
- Hence, *the decoding of a prefix can be accomplished as soon as the binary sequence representing a source symbol is fully received*.
- For this reason, prefix codes are also referred as *instantaneous codes*.

Average Codeword Length by using Kraft-McMillan Inequality (1/6)

- The average codeword length $\overline{(L)}$ can be bounded as follows:

$$H(S) \leq \overline{L} < H(S) + 1$$

- For a prefix code, $p_k = 2^{-l_k}$

Average Codeword Length by using Kraft-McMillan Inequality (2/6)

- The average codeword length is given by,

$$\overline{L} = \sum_{k=0}^{K} p_k l_k$$

- Let us substitute the value of p_k in the formula for $\overline{(L)}$

Average Codeword Length by using Kraft-McMillan Inequality (3/6)

- Now, the average codeword length is

$$\overline{L} = \sum_{k=0}^{K-1} 2^{-l_k} l_k = \sum_{k=0}^{K-1} \frac{l_k}{2^{l_k}} \quad Eqn.(1)$$

- The entropy of the source is given by,

$$H(S) = \sum_{k=0}^{K-1} p_k \log_2 \frac{1}{p_k}$$

Average Codeword Length by using Kraft-McMillan Inequality (4/6)

- Let us substitute the value of p_k in the formula for H(S)

$$H(S) = \sum_{k=0}^{K-1} 2^{-l_k} \log_2 \frac{1}{2^{-l_k}}$$

$$H(S) = \sum_{k=0}^{K-1} 2^{-l_k} \log_2 2^{l_k}$$

Average Codeword Length by using Kraft-McMillan Inequality (5/6)

- Now, the entropy, H(S)

$$H(S) = \sum_{k=0}^{K-1} l_k 2^{-l_k} \log_2 2$$

$$H(S) = \sum_{k=0}^{K-1} l_k 2^{-l_k}$$

$$H(S) = \sum_{k=0}^{K-1} \frac{l_k}{2^{l_k}} \quad Eqn.(2)$$

Average Codeword Length by using Kraft-McMillan Inequality (6/6)

- By comparing Eqn. (1) and Eqn. (2), For a prefix code that satisfies an efficient Kraft-McMillan Inequality,

$$\overline{L} = H(S)$$

- Kraft-McMillan inequality provides a necessary condition for uniquely decodable codes. That is, if a code is uniquely decodable, the codeword lengths have to satisfy the inequality.